📘 DOCUMENTACIÓN TÉCNICA Y DE APRENDIZAJE: TRAVEL CC

Proyecto: INGT1003 - Arquitectura de Desarrollo (2026-2)
Equipo: Leandro Manríquez, Ignacia Quiroz
Lema: "La ruta que aprende"

📋 Tabla de Contenidos

1. Introducción al Proyecto
2. ¿Qué es Flutter y por qué lo usamos?
3. Conceptos Fundamentales de Dart
4. Arquitectura del Proyecto (MVVM + Clean Architecture)
5. Estructura de Carpetas Explicada
6. Anatomía de un Widget en Flutter
7. Análisis Archivo por Archivo
8. Sistema de Navegación con GoRouter
9. Principios de Diseño Aplicados (DRY y SOLID)
10. Glosario de Términos Técnicos

---

1. Introducción al Proyecto

TRAVEL CC es una aplicación móvil diseñada para el transporte pesado de carga. Resuelve un problema crítico: los mapas convencionales (como Google Maps o Waze) están diseñados para vehículos livianos e ignoran restricciones físicas (altura, peso, ancho) y normativas (permisos de vialidad desde 4,2 m).

Nuestra propuesta de valor se basa en 3 pilares:

1. **Navegación Heavy:** Rutas calculadas según las dimensiones reales del vehículo.
2. **Fleet Tracking:** Monitoreo en tiempo real para jefes de flota.
3. **Mapa que Aprende (Crowdsourcing):** Los conductores reportan desvíos y anomalías, mejorando la base de datos geoespacial colectivamente (validado con datos de CONASET: 2.934 colisiones con camiones en 2024).

---

2. ¿Qué es Flutter y por qué lo usamos?

Flutter es un framework de UI de código abierto creado por Google. Permite compilar aplicaciones nativas para móviles (iOS/Android), web y escritorio desde una única base de código escrita en el lenguaje Dart.

¿Por qué lo elegimos para TRAVEL CC?

- **Multiplataforma real:** Un solo desarrollo para conductores que usan Android y jefes de flota que usan iOS.
- **Rendimiento nativo:** Compila a código máquina, logrando 60 FPS, crucial para una app de navegación en movimiento.
- **UI consistente:** Permite replicar exactamente el diseño oscuro y de alto contraste definido en nuestro proyecto.

---

3. Conceptos Fundamentales de Dart

Dart es el lenguaje detrás de Flutter. Conceptos clave que usamos:

- **Tipado estático con inferencia:** El tipo se declara o se infiere, evitando errores en tiempo de ejecución.
- **Null Safety:** Dart obliga a manejar valores nulos explícitamente, evitando el temido "NullPointerException".
- **Asincronía (Future/async/await):** Fundamental para simular llamadas a red (como el login).
- **Clases y Objetos:** Usamos clases para los ViewModels, que gestionan el estado de la pantalla.

---

4. Arquitectura del Proyecto

Aplicamos una combinación de Clean Architecture y el patrón MVVM (Model-View-ViewModel) para garantizar que el código sea escalable, testeable y mantenible.

El Patrón MVVM en TRAVEL CC:

- **Model:** Estructura de los datos puros (ej. datos simulados de camiones activos).
- **View:** La interfaz de usuario (UI). Lo que se ve. Solo contiene Widgets.
- **ViewModel:** El "cerebro". Conecta Model y View. Maneja el estado y la lógica.

Regla de oro: La View nunca debe contener lógica de negocio compleja. Solo "escucha" al ViewModel y se reconstruye cuando este avisa que algo cambió.

---

5. Estructura de Carpetas Explicada

Organizamos el código por funcionalidades (features), no por tipo de archivo.

- **main.dart:** Punto de entrada de la app.
- **core/theme/:** Centralización de estilos (colores, tipografía, espaciado) para aplicar el principio DRY.
- **features/:** Módulos independientes (home, auth, dashboard, map). Cada uno tiene su propia carpeta de `viewmodels` y `views`.
- **routes/:** Configuración centralizada de navegación.

---

6. Anatomía de un Widget en Flutter

En Flutter, todo es un Widget. Son como piezas de LEGO.

- **StatelessWidget:** No cambia con el tiempo (ej. un texto estático).
- **StatefulWidget:** Puede cambiar. En MVVM, delegamos este estado al ViewModel para mantener la View lo más simple posible.

Widgets de Layout usados:

- **Scaffold:** Estructura base de la pantalla.
- **Column / Row:** Organizan elementos vertical u horizontalmente.
- **Stack:** Apila elementos uno sobre otro (usado en el mapa).
- **Container / SizedBox:** Cajas para dar espacio, color o bordes.

---

7. Análisis Archivo por Archivo

- **main.dart:** Es el director. Inicializa la app, aplica el tema oscuro y conecta el sistema de navegación.
- **app_colors.dart:** Centraliza todos los colores. Si el cliente pide cambiar el naranja por azul, solo modificamos esta línea y toda la app se actualiza.
- **auth_viewmodel.dart:** Contiene la lógica del login. Usa controladores para leer los inputs y simula una carga asíncrona.

- **dashboard_view.dart:** Muestra las estadísticas. Utiliza un "Consumer" para reconstruir solo la parte de la UI que depende de los datos.

---

8. Sistema de Navegación con GoRouter

Usamos el paquete `go_router` para una navegación declarativa y segura.

Diferencia clave aprendida:

- `context.goNamed('login')`: Reemplaza la pantalla actual. Ideal para flujos lineales (Inicio -> Login) donde no queremos que el botón "Atrás" regrese al inicio.
- `context.pushNamed('detalle')`: Apila la pantalla. Permite volver atrás con `context.pop()`.

---

9. Principios de Diseño Aplicados

1. **DRY (Don't Repeat Yourself):** No escribimos colores o espaciados al azar. Usamos variables centralizadas.
2. **Single Responsibility (SOLID):** HomeView solo dibuja. HomeViewModel solo piensa. Si hay un bug, sabemos exactamente dónde buscar.
3. **Separation of Concerns:** La lógica de negocio está aislada de la UI.

---

10. Glosario de Términos Técnicos

- **Widget:** Unidad básica de construcción de UI en Flutter.
- **ChangeNotifier:** Clase de Flutter que avisa a la View cuando sus datos internos cambian.
- **Consumer:** Widget que "escucha" a un ChangeNotifier y reconstruye su hijo cuando hay cambios.
- **Hot Restart (R en terminal):** Reinicia la app desde cero aplicando todos los cambios de código.
- **Mock Data:** Datos simulados usados en el Hito 1 hasta que el Backend esté listo.
- **Crowdsourcing:** Obtención de información de manera distribuida entre los usuarios (conductores).

🚀 Próximos Pasos (Hito 2)

1. Reemplazar los Mock Data del Dashboard con llamadas reales a una API REST.
2. Integrar un mapa real (Mapbox o Google Maps SDK for Flutter).
3. Implementar autenticación segura con JWT.
4. Desplegar la app en un dispositivo real.